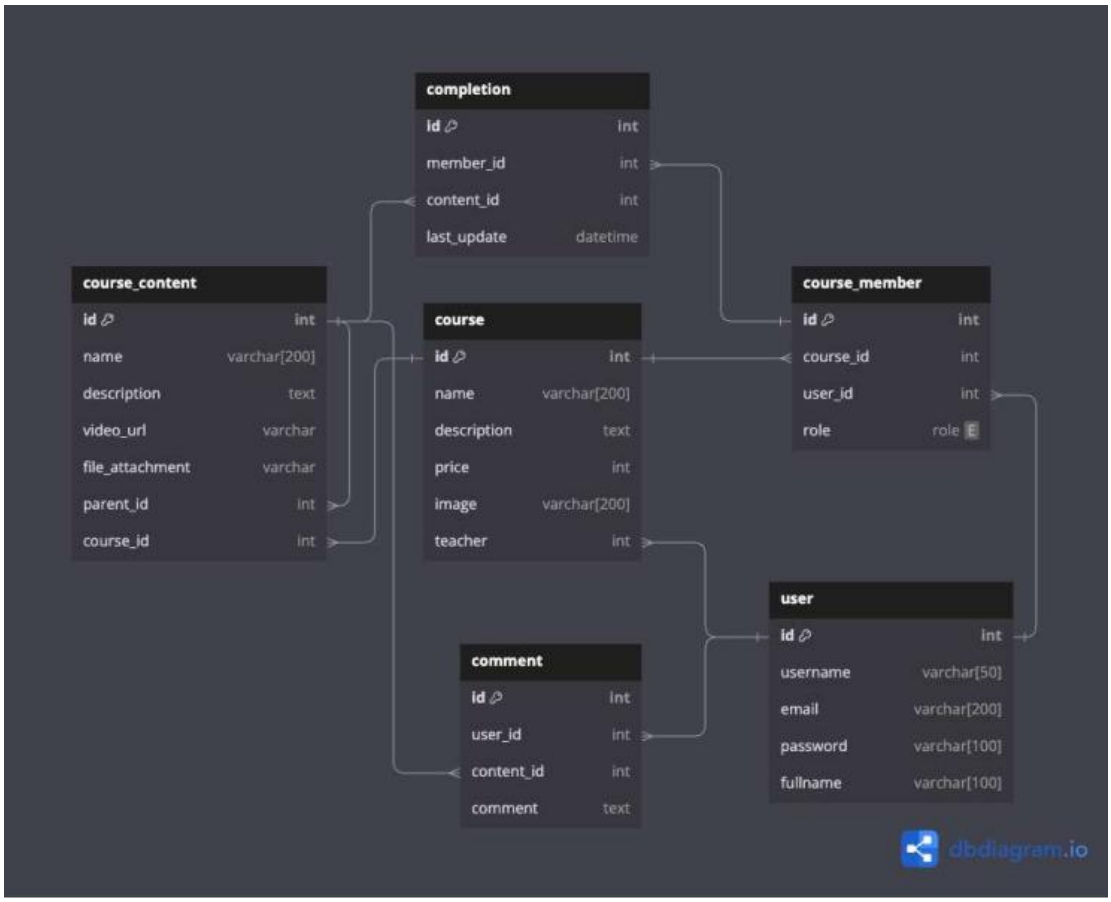
Pertemuan 6 : Object Relational Mapping (ORM)

1. Dari latihan sebelumnya Simple_LMS, buat model dari schema berikut :



2. Eksekusi Query dasar pada materi 6.
3. Import banyak data dengan import CVS (materi 6).
4. Eksekusi Query Relational pada materi 6.

Copy folder proyek latihan5, ubah menjadi proyek latihan6 :

Update requerements.txt :

```
django
psycopg2-binary
pillow
```

Update model.py :

```
from django.db import models
from django.contrib.auth.models import User

class Course(models.Model):
    name = models.CharField("nama matkul", max_length=100)
    description = models.TextField("deskripsi", default='-
  ')
    price = models.IntegerField("harga", default=10000)
       image   =   models.ImageField("gambar",   null=True,
  blank=True)
       #image   =   models.TextField("gambar",   null=True,
  blank=True)
            teacher      =      models.ForeignKey(User,
  verbose_name="pengajar", on_delete=models.RESTRICT)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        verbose_name = "Mata Kuliah"
        verbose_name_plural = "Mata Kuliah"

    def __str__(self) -> str:
        return self.name+" : "+str(self.price)

ROLE_OPTIONS = [('std', "Siswa"), ('ast', "Asisten")]

class CourseMember(models.Model):
           course_id     =     models.ForeignKey(Course,
  verbose_name="matkul", on_delete=models.RESTRICT)
            user_id      =      models.ForeignKey(User,
  verbose_name="siswa", on_delete=models.RESTRICT)
      roles   =   models.CharField("peran",   max_length=3,
  choices=ROLE_OPTIONS, default='std')
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
```

```
        verbose_name = "Subscriber Matkul"
        verbose_name_plural = "Subscriber Matkul"

    def __str__(self) -> str:
        return str(self.course_id)+" : "+str(self.user_id)

class CourseContent(models.Model):
          name    =    models.CharField("judul    konten",
  max_length=200)
    description = models.TextField("deskripsi", default='-
  ')
         video_url    =    models.CharField('URL    Video',
  max_length=200, null=True, blank=True)
    file_attachment = models.FileField("File", null=True,
  blank=True)
           course_id     =     models.ForeignKey(Course,
  verbose_name="matkul", on_delete=models.RESTRICT)
           parent_id     =     models.ForeignKey("self",
  verbose_name="induk",
                                on_delete=models.RESTRICT,
  null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        verbose_name = "Konten Matkul"
        verbose_name_plural = "Konten Matkul"

    def __str__(self) -> str:
        return '['+str(self.course_id)+"] "+self.name

class Comment(models.Model):
        content_id    =    models.ForeignKey(CourseContent,
  verbose_name="konten", on_delete=models.CASCADE)
         member_id    =    models.ForeignKey(CourseMember,
  verbose_name="pengguna", on_delete=models.CASCADE)
    comment = models.TextField('komentar')
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        verbose_name = "Komentar"
        verbose_name_plural = "Komentar"

    def __str__(self) -> str:
              return   "Komen:   "+self.content_id.name+"-
  "+self.member_id.user_id.username
```

Docker Compose Up :

```
docker compose up -d –build
```

Migrasi skema :

```
docker exec simplelms python manage.py makemigrations
```

Eksekusi Skrip Migrasi:

```
docker exec simplelms6 python manage.py migrate
```

Buat file core/urls.py :

```
from django.urls import path
from . import views

urlpatterns = [
   path('core/', views.core, name='core'),
   path("insert-user/", views.insert_user, name="insert_user"),
   path("insert-course/", views.insert_course, name="insert_course"),
   path("users/", views.user_list, name="user_list"),
   path("users/json/", views.user_json, name="user_json"),
   path("users/<int:pk>/", views.user_detail, name="user_detail"),
   path("search-user/", views.search_user, name="search_user"),
path("update-user/<int:pk>/", views.update_user, name="update_user"),
path("delete-user/<int:pk>/", views.delete_user, name="delete_user"),
path("delete-courses/",views.delete_all_courses,
  name="delete_all_courses"),
path(
        "delete-users-except/<int:pk>/",
        views.delete_users_except,
        name="delete_users_except",
    ),
]
```

Update file simplelms/urls.py :

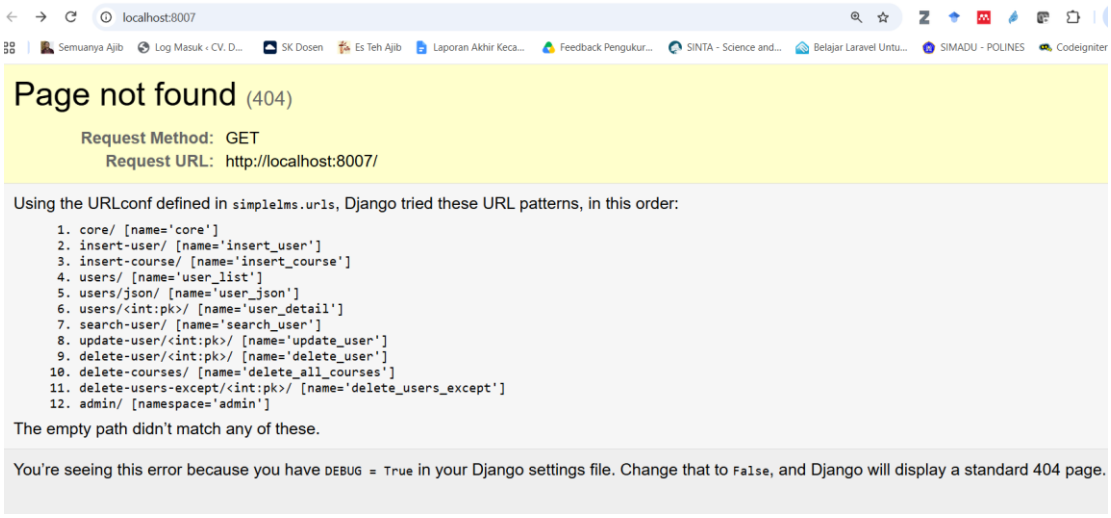
```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('', include('core.urls')),
    path('admin/', admin.site.urls),
]
```

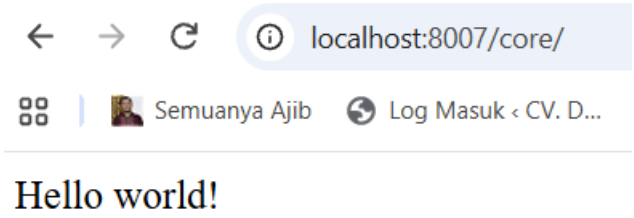
Docker compose Down - Up :

```
docker compose down
docker compose up -d
```

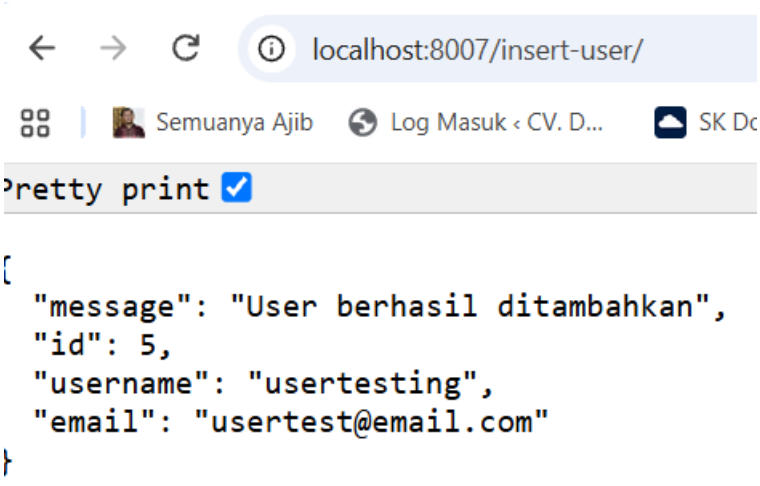
Buka Container di Docker / Browser :



`Localhost:8007/core (port disesuikan)`

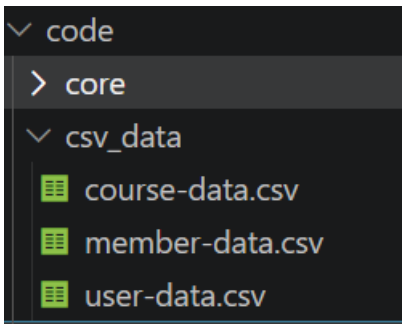


`Localhost:8007/insert-user (port disesuikan)`



Import File CSV :

Siapkan file CSV yang disimpan di folder code/csv_data.



course-data.cvs :

```
name,description,price,teacher
Ilmu Falak,Belajar Ilmu Falak ya,20000,2
Ghorib,Belajar ghorib ya,25000,3
Ilmu Es Teh,Belajar Ilmu Es Teh ya,30000,4
```

member-data.cvs :

```
course_id,user_id,roles
1,2,std
2,1,std
3,1,std
```

user-data.cvs :

```
username,password,email
Ghiyatsi,12345,ghiyatsi@gmail.com
Najwa,12345,najwa@gmail.com
Puji,12345,pujilestari2728@gmail.com
```

Buat file importer.py di folder yang sama dengan manage.py :

```
import os
import sys
sys.path.append(os.path.abspath(os.path.join(__file__,
  *[os.pardir] * 3)))
os.environ['DJANGO_SETTINGS_MODULE']                    =
  'simplelms.settings'
import django
django.setup()

import csv
```

```
from django.contrib.auth.models import User
from core.models import Course, CourseMember

with open('./csv_data/user-data.csv') as csvfile:
        reader = csv.DictReader(csvfile)
        for num, row in enumerate(reader):
                                                if       not
  User.objects.filter(username=row['username']).exists():
                User.objects.create_user(
                     id=num+2, username=row['username'],
                     password=row['password'],
                     email=row['email'])

with open('./csv_data/course-data.csv') as csvfile:
    reader = csv.DictReader(csvfile)
    for num,row in enumerate(reader):
                                              if          not
  Course.objects.filter(pk=num+1).exists():

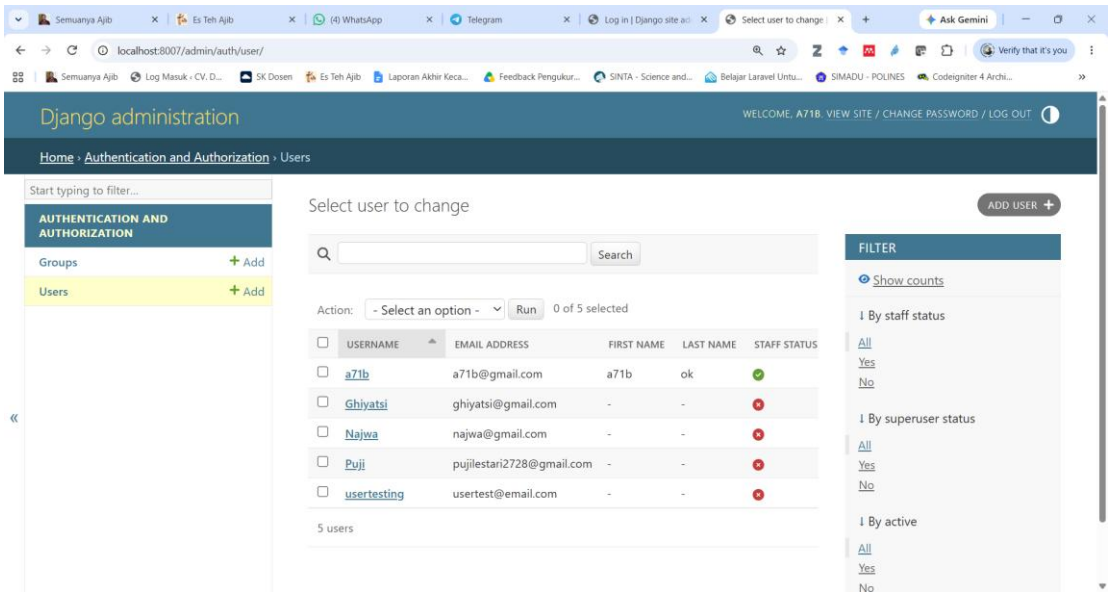
            Course.objects.create(
                    id=num+1, name=row['name'],
                    description=row['description'],
                    price=row['price'],
                     teacher=User.objects.get(pk=int(row['
  teacher'])))

with open('csv_data/member-data.csv') as csvfile:
    reader = csv.DictReader(csvfile)
    for num, row in enumerate(reader):
                                              if          not
  CourseMember.objects.filter(pk=num+1).exists():
            CourseMember.objects.create(
                     course_id=Course.objects.get(pk=int(r
  ow['course_id'])),
                     user_id=User.objects.get(pk=int(row['
  user_id'])),
                    id=num+1, roles=row['roles'])
```

Eksekusi di terminal :

```
docker exec simplelms python importer.py
```

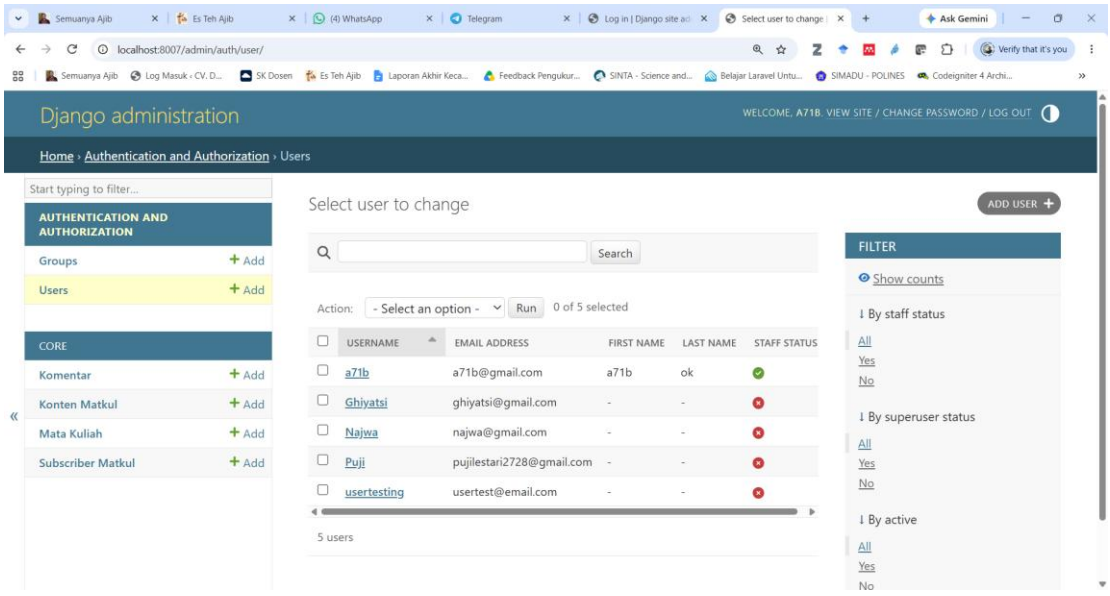
Cek di halaman admin :



Update file core/admin.py :

```
from django.contrib import admin
from .models import User, Course, CourseMember, CourseContent,Comment
# Register your models here.
admin.site.register(CourseMember)
admin.site.register(Course)
admin.site.register(CourseContent)
admin.site.register(Comment)
```

Refresh Halaman Admin :



Query Relational

Tambahkan di views.py :

```
#query relational
def allCourse(request):
    allCourse = Course.objects.all()
    result = []

    for course in allCourse:
        record = {
            'id': course.id,
            'name': course.name,
            'description': course.description,
            'price': course.price,
            'teacher': {
                'id': course.teacher.id,
                'username': course.teacher.username,
                'email': course.teacher.email,
                 'fullname': f"{course.teacher.first_name}
  {course.teacher.last_name}"
            }
        }
        result.append(record)

    return JsonResponse(result, safe=False)

def userCourses(request):
    user = User.objects.get(pk=3)
    courses = Course.objects.filter(teacher=user.id)

    course_data = []
    for course in courses:
        record = {
            'id': course.id,
            'name': course.name,
            'description': course.description,
            'price': course.price
        }
        course_data.append(record)

    result = {
        'id': user.id,
        'username': user.username,
        'email': user.email,
       'fullname': f"{user.first_name} {user.last_name}",
        'courses': course_data
    }
```

```
    return JsonResponse(result, safe=False)

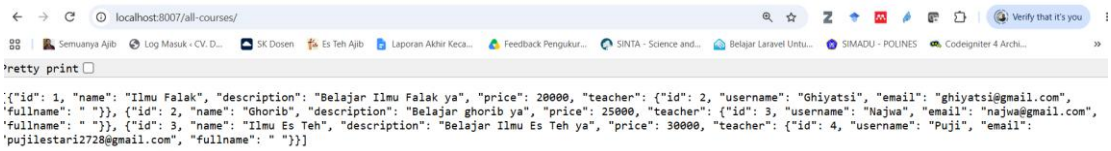
def courseStat(request):
    courses = Course.objects.all()
    stats = courses.aggregate(max_price=Max('price'),
                              min_price=Min('price'),
                              avg_price=Avg('price'))
     result = {'course_count': len(courses), 'courses':
  stats}
    return JsonResponse(result, safe=False)

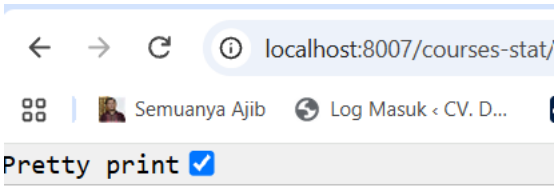
def courseMemberStat(request):
                                    courses                =
  Course.objects.filter(description__contains='python') \
                             .annotate(member_num=Count('c
  oursemember'))
    course_data = []
    for course in courses:
         record = {'id': course.id, 'name': course.name,
  'price': course.price,
                 'member_count': course.member_num}
        course_data.append(record)
     result = {'data_count': len(course_data), 'data':
  course_data}
    return JsonResponse(result)
```

Tambahkan di urls.py :

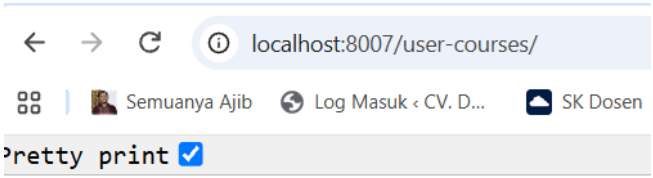
```
path("all-courses/", views.allCourse, name="all_courses"),
path("user-courses/", views.userCourses, name="user_courses"),
path("courses-stat/", views.courseStat, name="courses_stat"),
path("courses-member-stat/",           views.courseMemberStat,
  name="courses_member_stat"),
```

Cek hasil di browser :

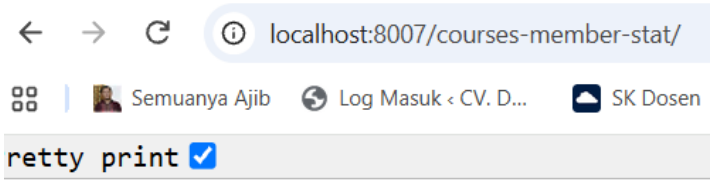





```
{
  "course_count": 3,
  "courses": {
    "max_price": 30000,
    "min_price": 20000,
    "avg_price": 25000
  }
}
```


```
{
  "id": 3,
  "username": "Najwa",
  "email": "najwa@gmail.com",
  "fullname": " ",
  "courses": [
    {
      "id": 2,
      "name": "Ghorib",
      "description": "Belajar ghorib ya",
      "price": 25000
    }
  ]
}
```


```
  "data_count": 1,
  "data": [
    {
      "id": 4,
      "name": "Python",
      "price": 50000,
      "member_count": 1
    }
  ]
```